

1 Metodología

Con base en los detalles de diseño especificados en clase se implementaron dos versiones de un algoritmo de solución, en una la representación usada fue hecha con matrices binarias y la otra con vectores de permutación.

1.1 Detalles técnicos: Representación de Matrices Binarias

Teniendo en cuenta la complejidad computacional de esta implementación se tomaron decisiones de esta versión que no estuvieron orientadas a afectar el espacio de búsqueda pero si a agilizar algunos procesos que del diseño del algoritmo genético se desprenden. Las alteraciones que varían en comparación al diseño original se describen a continuación.

1. **Función de evaluación:** Para el cálculo de los ataques entre reinas se considera el hecho siguiente. Es demostrable que en esta representación dadas las posiciones (i, j) y (r, s) de dos reinas, estas se encontraran en conflicto o bien habrá un ataque entre ellas si es que $|i - r| = |j - s|$ con $i \neq r$ ó bien es el caso que $j \neq s$. De igual manera habrá un ataque entre reinas si es que (en esta implementación) $i = r$, en este caso no es necesario revisar los valores j, s por los detalles del siguiente punto. Esta idea puede extrapolarse a la representación de vectores de permutación pero se abordará este hecho en un momento.
2. **Construcción de individuos y función de reparación:** Como se comentó al inicio, en esta implementación se tomaron algunas libertades creativas con el fin de agilizar la ejecución de procesos, particularmente el proceso para generar individuos iniciales se lleva a cabo partiendo de una matriz de ceros, sobre la que se recorrerá cada una de sus columnas y se elegirá una posición aleatoria dentro de cada una de estas donde colocar la reina. Generar una solución siguiendo esta regla obliga ciertamente a que en las soluciones no exista la posibilidad de encontrar mas de una reina por columna. Y si bien esta decisión podría considerarse sesgada, el hecho es que los experimentos realizados demuestran un gran desempeño de este algoritmo usando esta adecuación. En resumidas palabras esta modificación permite al algoritmo evitar casos en los que las reinas se apiñonan en una sola columna. Siguiendo esta idea, se implemento una función de reparación que ciertamente no estaba considerada en el diseño original de esta representación. La función desempeña el papel de impedir que mas de una reina se coloque en una sola columna, moviendo a las reinas extra que por columna se encontrasen a otras columnas que sin reina se observasen.

1.2 Detalles técnicos: Representación de permutaciones

Esta versión resultó con una menor complejidad de programación, inclusive como se verá en los experimentos de la siguiente sección es evidente que la convergencia de las soluciones se ve ampliamente beneficiada.

1. **Función de evaluación:** La función en este caso también cuenta cuantos ataques pueden encontrarse entre reinas, sin embargo, tiene el beneficio de que no se encontrarán casos en los que haya mas de una reina por fila o por columna. Aplicando el mismo principio de la representación de matrices se puede considerar a la posición de un par de reinas en el vector de solución i, r como las filas del tablero en el que se encuentran, y al valor que toman $j, s \in 1, 2, \dots, 8$ como la columna del vector en el que se encuentran. Análogamente, se considera que existe un ataque entre reinas si es que $|i - r| = |j - s|$ con $i \neq r$ ó bien es el caso que $j \neq s$.

2 Experimentación y conclusiones

Para el proceso de experimentación se dieron 30 semillas distintas (en este caso valores enteros de 1 a 30). Se guardo el menor numero de ataques por evaluación y de estos valores se encontró el fitness del peor individuo, el mejor, la media, la mediana y la desviación estándar de los datos que se poseían. Se muestra también la convergencia de la semilla No.15 de cada implementación

Estadístico	AlgPerm	AlgoMat
Mejor	0.00	0.00
Media	0.00	0.67
Mediana	0.00	1.00
Desv. Stdr	0.00	0.55
Peor	0.00	2.00

Table 1: En esta se puede inferir que en la versión de permutaciones se alcanza el optimo en to en comparación a la versión de matrices que si bien alcanza el optimo no lo hace en todas las evaluaciones

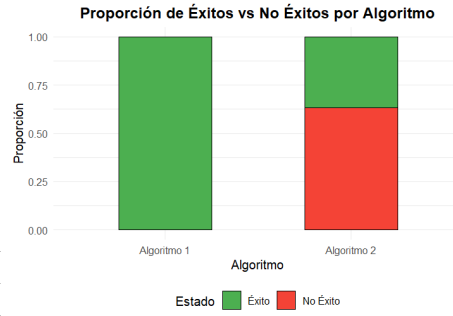


Fig. 1: Proporción de Éxitos vs No Éxitos

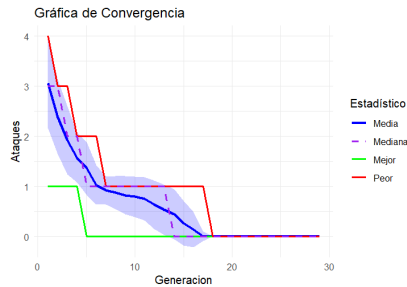


Fig. 2: Convergencia de la evaluación implementación de permutaciones

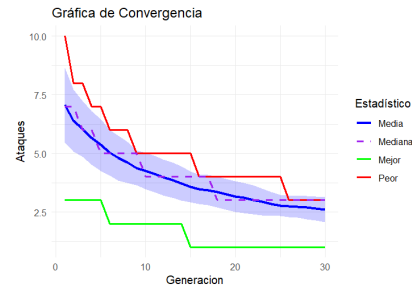


Fig. 3: Convergencia de la evaluación implementación de matrices